E.E.S.T. N°5

El Guardapolvo · Técnica Run

Documentación técnica del juego y del repositorio

Integrantes: Alexis Vallejos, Darian Seselovsky, Nacho Torres y Tiziano Riquelme

Proyecto para la Expo Técnica

Curso: 7° 4°

Fecha: 28 de octubre de 2026

Resumen

En este trabajo se documenta *El Guardapolvo* (también llamado *Técnica Run*), un juego de carrera en tercera persona ambientado en la avenida Juan B. Justo de Mar del Plata, que se controla con el cuerpo frente a una cámara web o con el teclado. Se describen la estructura del repositorio, el flujo del programa, el sistema de seguimiento corporal basado en MediaPipe Pose, la física y las colisiones, el modo historia, el escenario, la música y el proceso de modelado de los personajes con MakeHuman, Python y Blender. El seguimiento se probó dentro del juego real con una cámara virtual: se registraron la calibración, los cambios de carril, el salto, el agachado y una pérdida momentánea del jugador. También se informan los resultados de las pruebas automáticas y los problemas encontrados durante la revisión.

Palabras clave: seguimiento corporal, MediaPipe, Three.js, Web Worker, colisiones, modelado 3D, Blender, glTF.

Índice

Introducción 4
Desarrollo 4
1. Descripción general del juego 4
2. Estructura del repositorio 6
3. Arquitectura y flujo del programa 8
4. Seguimiento corporal (tracking) 10
4.1 Captura y detección del esqueleto 10
4.2 Elección del jugador 11
4.3 Calibración 12
4.4 Filtrado de los puntos 13
4.5 Detección de carril, salto y agachado 14
4.6 Brazos y manos 16
4.7 Sesión, cuenta atrás y pérdida del jugador 17
4.8 Prueba práctica del seguimiento 18
5. Jugabilidad, física y colisiones 21
6. Modo historia 25
7. Escenario: Juan B. Justo 27
8. Modelado de los personajes 28
8.1 Herramientas y flujo de trabajo 29
8.2 Estanga, el perseguidor 29
8.3 Personajes jugables 33
8.4 Interior de la escuela y casa 35
9. Sonido y música 35
10. Pruebas automáticas 36
11. Problemas encontrados y limitaciones 36
Conclusión 38
Referencias 39

Introducción

El proyecto nació como un corredor infinito controlado con el cuerpo: el jugador se para frente a la cámara, da un paso al costado para cambiar de carril, salta para esquivar tableros de dibujo y se agacha para pasar por debajo de bancos rotos. Con el tiempo se le sumaron un perseguidor (el profesor Estanga), personajes seleccionables, una ambientación marplatense, un menú propio, un modo historia con videos y música.

El objetivo de esta documentación es explicar cómo está hecho el juego, con capturas del juego, capturas del código con su número de línea y evidencia de las pruebas. El seguimiento corporal recibe la mayor atención porque es la parte que el usuario final ve y usa todo el tiempo. Además se explica cómo se modelaron los personajes.

Todo lo que se presenta como resultado se obtuvo sobre el repositorio en la revisión `b7ca369` (28 de octubre de 2026). Las capturas se tomaron con Google Chrome y Chromium controlados por Playwright en una Mac con procesador Intel i5 y GPU integrada Intel UHD 617. Cuando una prueba no se pudo hacer en condiciones reales, se aclara.

Desarrollo

1. Descripción general del juego

El juego es una aplicación web estática: un archivo `index.html`, módulos de JavaScript y recursos (modelos GLB, texturas, videos y música). No necesita compilación ni servidor propio; alcanza con servirlo por HTTP local o publicarlo en GitHub Pages. La escena 3D se dibuja con Three.js 0.160, que está copiado en `vendor/three`.

Como se observa en la Figura 1, el menú principal ofrece dos modos. En el **modo infinito** se corre por Juan B. Justo hasta chocar; la puntuación es la distancia recorrida más diez puntos por moneda. En el **modo historia** el alumno se olvidó el guardapolvo: corre hasta su casa, lo busca y vuelve a la Técnica. El menú lateral da acceso a Ajustes, Récords, Personajes y Logros.

Figura 1

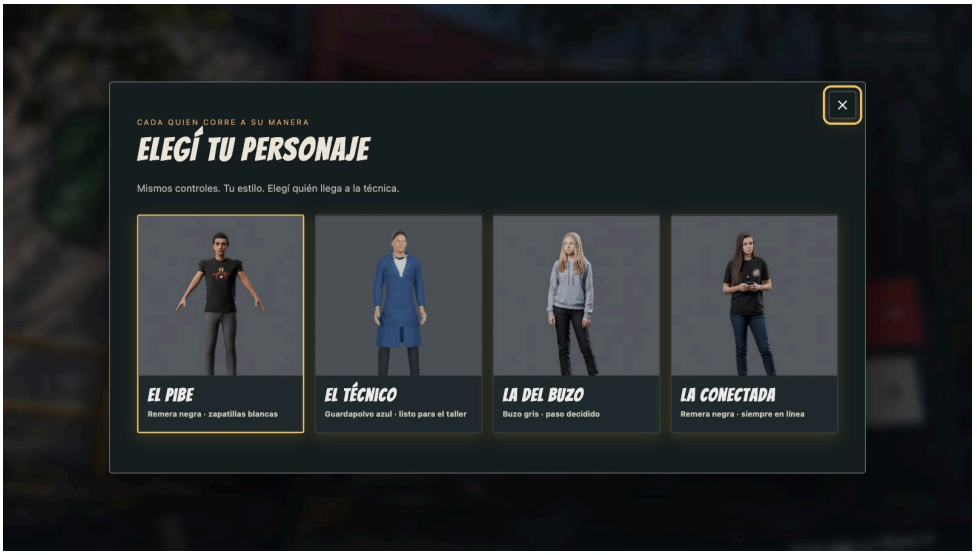
Menú principal del juego con los dos modos y las opciones laterales.



El jugador elige entre cuatro personajes (Figura 2). La elección se guarda en el navegador con `localStorage` y la página se recarga con el modelo elegido. Todos usan los mismos controles y el mismo esqueleto.

Figura 2

Selector de personajes: El pibe, El técnico, La del buzo y La conectada.



Los controles se resumen en la Tabla 1. El teclado y la cámara pueden convivir: si se presiona una tecla de juego, la sesión pasa a modo teclado y deja de obedecer a la cámara hasta que se vuelva a activar.

Tabla 1

Controles del juego con cámara y con teclado.

Acción	Con el cuerpo (cámara)	Con el teclado
Cambiar de carril	Paso a la izquierda o a la derecha	← / → o A / D
Saltar	Saltar (sube cadera, hombros y pies)	↑, W o Espacio
Agacharse	Bajar la cabeza o encoger el torso	↓ o S (mantener)
Empezar / reiniciar	Levantar las dos manos por encima de la nariz	Enter o botón
Pausa	Automática si se pierde al jugador	P o Esc
Recalibrar	—	C
Modo depuración	—	D (empieza sin cámara ni cuenta atrás)

Nota. Datos tomados de `main.js` (manejo de teclado y de gestos) y de `pose.js` (detección de gestos).

En Ajustes (Figura 3) se activa la cámara, se elige la calidad gráfica, el volumen y la sensibilidad de salto y agachado. La sensibilidad divide los umbrales de detección: con 120 % hace falta moverse menos para que el juego registre el gesto.

Figura 3

Ventana de ajustes con volumen, instrucciones, calidad gráfica, sensibilidad y botón para activar la cámara.



2. Estructura del repositorio

El repositorio tiene los módulos del juego en la raíz, los recursos en `assets/`, las bibliotecas externas en `vendor/`, los scripts de modelado en `tools/`, las pruebas en `tests/` y la documentación interna en `docs/`. La Tabla 2 resume para qué sirve cada módulo principal.

Tabla 2

Módulos principales del juego y su función.

Archivo	Líneas	Función
`main.js`	268	Punto de entrada: crea el juego, la sesión, el modo historia, la música y el ranking; conecta teclado, cámara y botones; bucle de cuadros.
`game.js`	997	Clase Game: escena Three.js, generación de obstáculos y monedas, física, colisiones, perseguidor, calidad gráfica.
`collisions.js`	37	Cálculo del intervalo de contacto con obstáculos dentro de cada cuadro.
`session.js`	56	PlaySession: cuenta atrás 3-2-1, pausa manual, pausa por pérdida del seguimiento y modo teclado.
`pose.js`	512	PoseController: cámara, selección y calibración del jugador, filtro One Euro, detección de carril, salto, agachado y manos arriba.
`pose-worker.js`	57	Web Worker que ejecuta MediaPipe Pose Landmarker fuera del hilo principal.
`arm-tracking.js`	29	Direcciones de brazo, antebrazo y mano a partir de los puntos de MediaPipe.
`character.js`	523	Carga de personajes GLB, niveles de detalle y animación procedural (carrera, salto, agachado, brazos).
`camera-rig.js`	87	Cámara en tercera persona: seguimiento, altura sobre colectivos y atenuación de lo que tapa.
`world.js`	697	Mundo: calzada, veredas, paredones, edificios, farolas y reciclado de tramos; recorrido invertido.
`avenue-*.js`	479	Recorrido de Juan B. Justo: orden de lotes, fachadas e hitos (estaciones, maderera, Técnica 5).
`street-textures.js / textures.js`	1049	Texturas procedurales dibujadas con canvas (asfalto, baldosas, hormigón, murales).
`bus.js / props.js / school-props.js`	542	Colectivos, objetos del escenario y obstáculos escolares (tablero y banco).
`story.js`	211	StoryMode: videos, ida a casa, recogida del guardapolvo, vuelta y final.
`music.js`	145	GameMusic: lista de reproducción aleatoria sin repeticiones.
`menu.js`	44	Menú, efectos de sonido sintetizados, selección de personaje y logros.
`leaderboard.js / ranking.js`	151	Ranking local de diez puntajes con iniciales, separado por modo.
`school-interior.js`	411	Interior de la escuela (vista previa, no integrado al juego).

Nota. Cantidad de líneas medida con `wc -l` en la revisión b7ca369.

El historial de Git muestra cómo creció el proyecto. La Tabla 3 agrupa los commits por etapa. En total hay 50 commits entre el 22 y el 28 de octubre de 2026, hechos por varios integrantes y en parte con asistentes de IA (Claude y Codex), que figuran como coautores en los mensajes.

Tabla 3

Etapas del desarrollo según el historial de commits.

Fecha	Etapa	Commits representativos
22/10	Base del juego y control con cámara	2420ac0 «Subway Surfer controlado con el cuerpo vía cámara»; 4419457 «Detectar mejor el salto y el agacharse»; c10fb4a «Seguir siempre al jugador, pausar si se pierde»
23/10	Tercera persona, guardapolvo, brazos y ambientación	e4c3c32; 004cbca «Corregir seguimiento, cuenta atrás y apoyo en vagones»; 1620238 «Reflejar brazos y manos de cámara»; b9f7f7f «Ambientar en Mar del Plata»
24–25/10	Profesor modelado, Juan B. Justo y ranking	c832bc7; 5c43577 «Integrar profesor modelado en Blender»; 070e2a9 «jb justo»; 55377d2 «Agregar ranking con iniciales»
26–27/10	Personajes, menú, Estanga y sombras	d804ced «Integrar teen-boy animado»; 3d9cc8d «Crear menú El Guardapolvo»; 759b169 «cargar MediaPipe localmente»; 9be9986 «modelados de jugadores y Estanga»
27–28/10	Modo historia, videos, música y diagramas	4738a13 «Agregar modo historia»; 6d3b16a «Corregir tropiezos, colisiones»; 777c220 «actualización automática de música»; b7ca369 «Pausar con Esc y recorrer Juan B. Justo al revés»

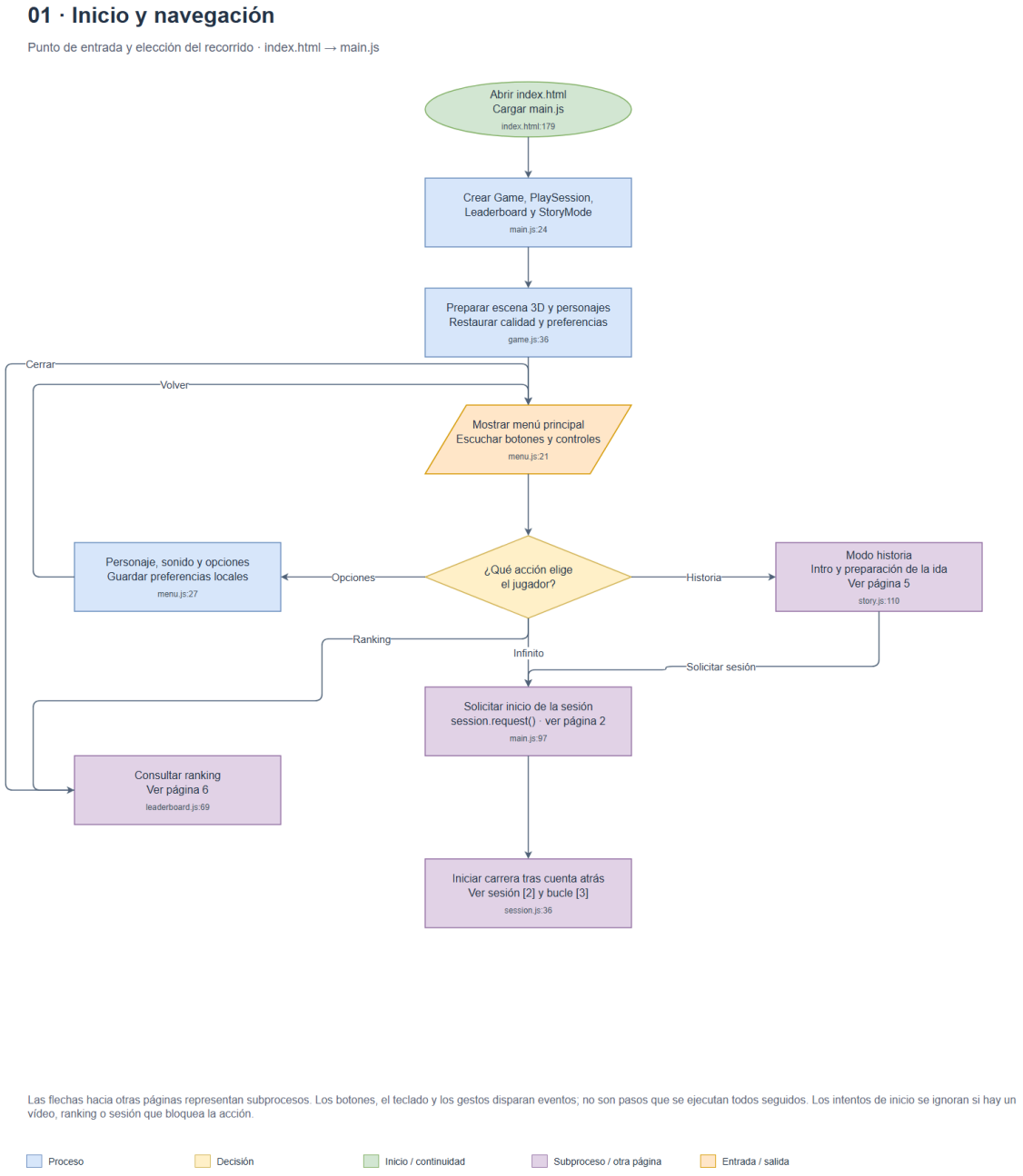
3. Arquitectura y flujo del programa

El punto de entrada es `main.js`. Crea el objeto `Game` (escena, física y dibujo), la sesión de juego `PlaySession` (cuenta atrás y pausas), el modo historia, el ranking y la música. Después arranca un bucle con `requestAnimationFrame` que, en cada cuadro, llama a `game.update(dt)` con el tiempo transcurrido. Mientras el menú está abierto el juego no se actualiza, para no gastar GPU detrás de una imagen fija.

El repositorio incluye un diagrama de flujo editable en draw.io (`docs/diagrama-flujo.drawio`) con seis páginas. La Figura 4 muestra la primera: carga, menú y elección de modo. Cada bloque indica el archivo y la línea de código que representa.

Figura 4

Diagrama de flujo de inicio y navegación (docs/diagrama-flujo/inicio.png).



La comunicación entre el seguimiento y el juego se hace con funciones de aviso (*callbacks*). `pose.js` no conoce al juego: solo avisa «cambió el carril», «saltó», «se agachó» o «se perdió al jugador». `main.js` recibe esos avisos y los traduce a llamadas de `Game` (Figura 5). Esto permite probar la detección sin la escena 3D.

Figura 5

Avisos del detector de poses conectados con el juego (main.js).

main.js — líneas 115–149

```
const poseHandlers = {
  onArms: targets => { if (!session.keyboard) game.setArmTargets(targets); },
  onStatus: (text) => {
    $('pose-status').textContent = text;
    if (!session.keyboard && session.pending && !session.ready) $('pause-text').textContent = text;
  },
  onLost: () => {
    session.tracking(false);
    if (pose && !pose.running) $('retryCam').classList.remove('hidden');
  },
  onCalibrating: () => session.tracking(false),
  onCalibrated: () => {
    flash('¡Listo! 🙌', '#7dff9a');
    if (!session.keyboard) game.setLane(1);
    session.tracking(true);
  },
  onLane: (lane) => {
    if (session.keyboard) return;
    game.setLane(lane + 1);
    flash(['⬅', '⬆', '➡'][lane + 1]);
  },
  onJump: () => {
    if (session.keyboard) return;
    game.jump();sound('jump');
    flash('SALTO ↑', '#ffd84a');
  },
  onDuck: () => {
    if (session.keyboard) return;
    game.roll();sound('duck');
    flash('AGACHADO ↓', '#4ad0ff');
  },
  onDuckHold: () => { if (!session.keyboard) game.holdRoll(); },
  onDuckEnd: () => { if (!session.keyboard) game.releaseRoll(); },
  onHandsUp: () => { if (!session.keyboard) play(); },
};
```

4. Seguimiento corporal (tracking)

El seguimiento corporal es lo que convierte la cámara web en un control. Funciona en cuatro pasos: capturar la imagen, detectar el esqueleto, elegir y calibrar al jugador, y convertir sus movimientos en gestos. Todo ocurre dentro del navegador: los píxeles de la cámara no se envían a ningún servidor.

4.1 Captura y detección del esqueleto

La cámara se abre con `navigator.mediaDevices.getUserMedia`, que pide permiso al usuario y solo funciona en un contexto seguro (HTTPS o `localhost`). Se pide una imagen de 640 × 480 a 60 cuadros por segundo con la cámara frontal.

Cada cuadro se analiza con **MediaPipe Pose Landmarker**, un modelo de Google que devuelve 33 puntos del cuerpo (nariz, hombros, codos, muñecas, caderas, rodillas, tobillos, etc.) con coordenadas normalizadas entre 0 y 1 y un valor de visibilidad. El modelo corre en un **Web Worker** (`pose-worker.js`), es decir, en un hilo separado, para que el juego no se trabe mientras se analiza la imagen. El cuadro se envía como `ImageBitmap` transferible, sin copiarlo.

Como se ve en la Figura 6, primero se intenta el modelo *full* con aceleración por GPU; si falla, se usa el modelo *lite* por CPU. Si el navegador no permite el worker, `pose.js` carga MediaPipe en el hilo principal como respaldo. Se piden hasta tres personas (`numPoses: 3`) para poder ignorar a quien pase por detrás. MediaPipe 0.10.14, sus archivos WASM y los dos modelos están copiados en `vendor/mediapipe` con sus sumas SHA-256, así que activar la cámara no depende de servidores externos.

Figura 6

Configuración e inicio de MediaPipe en el Web Worker, con alternativa GPU → CPU (pose-worker.js).

pose-worker.js — líneas 1–38

```
// Detección del esqueleto en un hilo aparte: así el juego no se congela mientras
// MediaPipe analiza cada fotograma de la cámara.
// Es un worker "clásico" (no de módulo) porque MediaPipe carga su WASM con importScripts.
const BASE = new URL('./vendor/mediapipe/', self.location.href).href.replace(/\/$/, '');
const MODEL = kind => `${BASE}/models/pose_landmarker_${kind}.task`;

// Detectamos varias personas para poder quedarnos con el jugador e ignorar
// a quien pase por detrás
const OPTIONS = {
  runningMode: 'VIDEO',
  numPoses: 3,
  minPoseDetectionConfidence: 0.6,
  minPosePresenceConfidence: 0.6,
  minTrackingConfidence: 0.6,
};

self.exports = {};
importScripts(`${BASE}/vision_bundle.cjs`);
const { FilesetResolver, PoseLandmarker } = self.exports;

let landmarker = null;

async function init() {
  const vision = await FilesetResolver.forVisionTasks(`${BASE}/wasm`);
  const create = (kind, delegate, extra = {}) =>
    PoseLandmarker.createFromOptions(vision, {
      baseOptions: { modelAssetPath: MODEL(kind), delegate },
      ...OPTIONS,
      ...extra,
    });
  try {
    landmarker = await create('full', 'GPU', { canvas: new OffscreenCanvas(1, 1) });
    return 'GPU';
  } catch {
    landmarker = await create('lite', 'CPU');
    return 'CPU';
  }
}
```

4.2 Elección del jugador

Si hay varias personas en cuadro, el juego tiene que saber cuál es el jugador. La función `selectPlayer` (Figura 7) hace lo siguiente: si todavía no hay jugador, elige a la persona más grande (la más cercana a la cámara) entre las que están cerca del centro; si ya hay jugador, elige a la que está más cerca de su posición anterior y tiene un tamaño parecido. Si el mejor candidato está demasiado lejos (más de 2,5 anchos de hombro), se considera que el jugador no está. Las demás personas se dibujan en gris y no cuentan.

Figura 7

Selección del jugador entre varias personas detectadas (pose.js).

pose.js — líneas 215–244

```
215  // De todas las personas detectadas, elige al jugador: la que sigue donde estaba
216  // el jugador en el fotograma anterior. Si aún no hay jugador, la más grande
217  // (la más cercana a la cámara) cerca del centro. Los demás se ignoran.
218  selectPlayer(poses) {
219    const cands = [];
220    for (const lm of poses) {
221      const v = (i) => (lm[i].visibility ?? 1) > 0.5;
222      const pts = [NOSE, L_SHOULDER, R_SHOULDER, L_HIP, R_HIP].filter(v);
223      if (pts.length < 2) continue;
224      const size = v(L_SHOULDER) && v(R_SHOULDER)
225        ? Math.hypot(lm[L_SHOULDER].x - lm[R_SHOULDER].x, lm[L_SHOULDER].y - lm[R_SHOULDER].y)
226        : this.track?.size;
227      if (!size) continue;
228      cands.push({ lm, size, cx: avg(pts.map((i) => lm[i].x)), cy: avg(pts.map((i) => lm[i].y)) });
229    }
230    if (!cands.length) return null;
231
232    let best;
233    const t = this.track;
234    if (t) {
235      const score = (c) => Math.hypot(c.cx - t.cx, c.cy - t.cy) / t.size + Math.abs(Math.log(c.size / t.size)) * 2;
236      best = cands.reduce((a, c) => (score(c) < score(a) ? c : a));
237      if (score(best) > MAX_JUMP_AWAY) return null;
238    } else {
239      const central = cands.filter((c) => c.cx > 0.15 && c.cx < 0.85);
240      best = (central.length ? central : cands).reduce((a, c) => (c.size > a.size ? c : a));
241    }
242    this.track = { cx: best.cx, cy: best.cy, size: t ? t.size + (best.size - t.size) * 0.3 : best.size };
243    return best.lm;
244  }
```

4.3 Calibración

Las medidas en píxeles cambian según la altura de la persona y su distancia a la cámara. Por eso el juego primero calibra: toma 20 cuadros con el jugador quieto y guarda como referencia la posición horizontal, la altura de hombros, cadera, nariz y tobillos, el ancho de hombros y el largo del torso (Figura 8). Si el jugador se mueve más de un cuarto del ancho de hombros, la cuenta vuelve a empezar. Una parte del cuerpo solo se usa como referencia si se vio en al menos el 70 % de los cuadros.

Todos los umbrales se expresan en unidades del propio cuerpo: el carril se mide en *anchos de hombro* y el salto en *largos de torso*. Así el mismo movimiento funciona igual con un chico cerca de la cámara que con un adulto más lejos. Durante el juego la referencia se adapta lentamente (2 % por cuadro) cuando el jugador está en reposo, para compensar si se acerca o se aleja un poco.

Figura 8

Calibración con 20 cuadros quietos (pose.js).

pose.js — líneas 467–496

```
467  calibrate(m) {
468    this.samples.push(m);
469    if (this.samples.length > CALIB_FRAMES) this.samples.shift();
470
471    const pick = (k) => this.samples.map((p) => p[k]).filter((v) => v != null);
472    const mean = (k) => {
473      const v = pick(k);
474      // Solo se usa una parte del cuerpo si se ha visto en la mayoría de los fotogramas
475      return v.length >= this.samples.length * 0.7 ? avg(v) : null;
476    };
477    const mx = avg(pick('x')), my = avg(pick('sy'));
478    const sw = avg(pick('sw'));
479    const spread = Math.max(...this.samples.filter((p) => p.x != null).map((p) => Math.hypot(p.x - mx, p.sy - my)));
480
481    // Hay que estar quieto: si te mueves mucho, se reinicia la cuenta
482    if (spread > 0.25 * sw) this.samples = [m];
483    const pct = Math.round((this.samples.length / CALIB_FRAMES) * 100);
484    this.status(`Calibrando… quédate quieto en el centro (${pct}%)`);
485
486    if (this.samples.length >= CALIB_FRAMES) {
487      const torso = mean('torso');
488      this.base = {
489        x: mx, sy: my, sw,
490        ny: mean('ny'), hy: mean('hy'), ay: mean('ay'),
491        torso: Math.max(torso ?? sw * 1.3, 0.05),
492      };
493      this.lane = 0;
494      this.h.onCalibrated?.();
495    }
496  }
```

4.4 Filtrado de los puntos

Los puntos que entrega el detector tiemblan aunque la persona esté quieta. Para suavizarlos se usa el **filtro One Euro** (Casiez et al., 2012), un filtro pasa bajos cuya frecuencia de corte aumenta con la velocidad: cuando la persona está quieta filtra mucho y elimina el temblor; cuando se mueve rápido filtra poco y casi no agrega retraso. En el código (Figura 9) se usan dos configuraciones: la posición horizontal se suaviza más para que el carril sea estable, y las alturas casi sin retraso para que el salto se detecte a tiempo.

Figura 9

Umbrales de detección y filtro One Euro (pose.js).

pose.js — líneas 17–54

```
// Umbrales (en unidades relativas al cuerpo, así no dependen de la distancia a la cámara).
// Los de salto y agacharse se dividen por la sensibilidad elegida en el menú.
const LANE_ENTER = 0.6;    // anchos de hombro desplazado para cambiar de carril
const LANE_EXIT = 0.35;    // histéresis para volver al centro
const JUMP_UP = 0.11;      // subida del cuerpo (en largos de torso) para saltar
const JUMP_EARLY = 0.05;   // ...o una subida menor pero rápida
const JUMP_SPEED = 1.2;    // largos de torso por segundo
const DUCK_DOWN = 0.38;    // bajada/encogimiento para agacharse ("hacerse bolita")
const DUCK_DEEP = 0.6;     // tan agachado que no hace falta confirmar
const DUCK_EXIT = 0.2;     // por debajo de esto se considera que te levantaste
const CALIB_FRAMES = 20;   // frames quieto necesarios para calibrar
const LOST_FRAMES = 10;    // ~1/3 s sin ver al jugador = lo hemos perdido (se pausa)
const FORGET_FRAMES = 60;  // ~2 s sin verlo: se elige de nuevo a la persona más cercana
const MAX_JUMP_AWAY = 2.5; // cuánto puede "saltar" el jugador entre fotogramas (en anchos de hombro)

const avg = (arr) => arr.reduce((a, b) => a + b, 0) / arr.length;

// Filtro "One Euro": quita el temblor de los puntos cuando estás quieto pero casi no
// añade retraso cuando te mueves rápido (la suavidad depende de la velocidad).
class OneEuro {
  constructor(minCutoff = 2.5, beta = 6, dCutoff = 1.5) {
    Object.assign(this, { minCutoff, beta, dCutoff, x: null });
  }
  static alpha(cutoff, dt) {
    return 1 / (1 + 1 / (2 * Math.PI * cutoff * dt));
  }
  filter(x, t) {
    if (this.x == null || t - this.t > 0.3) {
      Object.assign(this, { x, dx: 0, t });
      return x;
    }
    const dt = Math.max(t - this.t, 1e-3);
    this.t = t;
    this.dx += OneEuro.alpha(this.dCutoff, dt) * ((x - this.x) / dt - this.dx);
    this.x += OneEuro.alpha(this.minCutoff + this.beta * Math.abs(this.dx), dt) * (x - this.x);
    return this.x;
  }
}
```

4.5 Detección de carril, salto y agachado

Carril. Se calcula cuánto se desplazó el centro de los hombros respecto del punto calibrado, dividido por el ancho de hombros. Para pasar del centro a un costado hay que superar 0,6; para volver al centro hay que bajar de 0,35. Esa diferencia entre umbrales se llama **histéresis** y evita que el personaje oscile entre dos carriles cuando el jugador queda justo en el límite. La imagen se trata en espejo: moverse a la derecha propia mueve al personaje a la derecha de la pantalla.

Salto. Se promedia cuánto subieron la cadera y los hombros (y los pies si se ven), en largos de torso. Hay salto si la subida supera 0,11, o si supera 0,05 con una velocidad mayor a 1,2 largos de torso por segundo (detección temprana). Después de un salto hay que volver cerca del reposo para poder saltar de nuevo (Figura 10).

Figura 10

Cálculo del carril con histéresis y del salto (pose.js).

pose.js — líneas 378–430

```
// --- Carril (absoluto respecto al punto calibrado, con histéresis) ---
if (m.x != null) {
  const dx = (m.x - b.x) / b.sw; // + = a la derecha
  let lane = this.lane;
  if (lane === 0) {
    if (dx < -LANE_ENTER) lane = -1;
    else if (dx > LANE_ENTER) lane = 1;
  } else if (Math.abs(dx) < LANE_EXIT) {
    lane = 0;
  } else if (lane === -1 && dx > LANE_ENTER) {
    lane = 1;
  } else if (lane === 1 && dx < -LANE_ENTER) {
    lane = -1;
  }
  if (lane !== this.lane) {
    this.lane = lane;
    this.h.onLane?.(lane);
  }
}

// --- Salto: sube todo el cuerpo (cadera y hombros) y, si se ven, los dos pies ---
const bodyRise = [];
if (m.hy != null && b.hy != null) bodyRise.push((b.hy - m.hy) / T);
if (m.sy != null) bodyRise.push((b.sy - m.sy) / T);
let jump = bodyRise.length ? avg(bodyRise) : null;
if (jump != null && m.ay != null && b.ay != null) jump = (jump + (b.ay - m.ay) / T) / 2;

// --- Agacharse / hacerse bolita: baja la cabeza o los hombros, o se encoge el torso ---
const duckCues = [];
if (m.sy != null) duckCues.push((m.sy - b.sy) / T);
if (m.ny != null && b.ny != null) duckCues.push(((m.ny - b.ny) / T) * 0.85);
if (m.torso != null) duckCues.push((1 - m.torso / T) * 0.9);
const duck = duckCues.length ? Math.max(...duckCues) : null;

// Velocidad vertical del cuerpo (largos de torso por segundo, + = subiendo)
let vUp = 0;
if (jump != null) {
  if (this.prev && now > this.prev.t) vUp = ((jump - this.prev.jump) * 1000) / (now - this.prev.t);
  this.prev = { jump, t: now };
}
this.meter = { jump: Math.max(0, jump ?? 0), duck: Math.max(0, duck ?? 0) };

if (jump != null && this.jumpArmed && !this.ducking) {
  const big = jump > JUMP_UP / sens;
  // La detección temprana por velocidad no vale justo al levantarse de agacharse
  const early = jump > JUMP_EARLY / sens && vUp > JUMP_SPEED && now - this.lastDuckEnd > 300;
  if (big || early) {
    this.jumpArmed = false;
    this.lastJump = now;
    this.h.onJump?.();
  }
}
if (jump != null && jump < JUMP_EARLY / 2) this.jumpArmed = true;
```

Agachado. Se toma la mayor de tres señales: cuánto bajaron los hombros, cuánto bajó la nariz y cuánto se encogió el torso. Para evitar que el pequeño amague antes de saltar cuente como agachado, la señal tiene que superar 0,38 durante dos cuadros seguidos, salvo que sea muy profunda (más de 0,6). Se ignora durante medio segundo después de un salto para que el aterrizaje no se confunda (Figura 11).

Figura 11

Confirmación del agachado y salida (pose.js).

pose.js — líneas 432–449

```
432    // Agacharse: hay que mantenerlo 2 fotogramas (así el pequeño amago antes de
433    // saltar no cuenta), salvo que sea muy profundo. Se ignora el aterrizaje.
434    if (duck != null && now - this.lastJump > 500) {
435      if (!this.ducking) {
436        this.duckFrames = duck > DUCK_DOWN / sens ? this.duckFrames + 1 : 0;
437        if (this.duckFrames >= 2 || duck > DUCK_DEEP / sens) {
438          this.ducking = true;
439          this.h.onDuck?.();
440        }
441      } else if (duck < DUCK_EXIT / sens) {
442        this.ducking = false;
443        this.h.onDuckEnd?.();
444        this.duckFrames = 0;
445        this.lastDuckEnd = now;
446      } else {
447        this.h.onDuckHold?.(); // sigues agachado: el corredor sigue rodando
448      }
449    }
```

Además, si las dos muñecas quedan por encima de la nariz durante 400 ms, se dispara el gesto de «manos arriba», que inicia o reinicia la partida sin tocar el teclado.

4.6 Brazos y manos

El personaje copia la posición de los brazos del jugador. `arm-tracking.js` (Figura 12) calcula, para cada brazo, la dirección hombro → codo, codo → muñeca y muñeca → índice, solo con puntos de visibilidad mayor o igual a 0,6. Con los brazos abajo se conserva el braceo de la animación de carrera; al levantar una mano, ese brazo pasa gradualmente a control directo sin afectar al otro. Los dedos no se copian porque MediaPipe Pose no entrega articulaciones de dedos.

Figura 12

Cálculo de las direcciones de brazo, antebrazo y mano (arm-tracking.js).

arm-tracking.js — líneas 1–29

```
// MediaPipe Pose: hombro, codo, muñeca y centro aproximado de la mano.
// Mismo espejo que la vista de cámara; sin inventar articulaciones de dedos.
export function armTargets(landmarks, aspect = 4 / 3) {
  const valid = i => landmarks?.[i] && (landmarks[i].visibility ?? 1) >= 0.6 &&
    Number.isFinite(landmarks[i].x) && Number.isFinite(landmarks[i].y);
  const direction = (a, b) => {
    if (!valid(a) || !valid(b)) return null;
    const x = -(landmarks[b].x - landmarks[a].x) * aspect;
    const y = -(landmarks[b].y - landmarks[a].y);
    const length = Math.hypot(x, y);
    return length > 0.025 ? [x / length, y / length, 0] : null;
  };
  const targets = {};
  // La izquierda anatómica detectada se ve a la izquierda en el espejo.
  for (const [side, shoulder, elbow, wrist, index, pinky] of [
    ['Right', 11, 13, 15, 19, 17], ['Left', 12, 14, 16, 20, 18],
  ]) {
    const hip=side==='Right'?23:24;
    const torso=valid(hip)&&valid(shoulder)?landmarks[hip].y-landmarks[shoulder].y:0;
    // Con los brazos descansando conservar el braceo de carrera; levantar una
    // mano activa gradualmente su control directo sin bloquear el otro brazo.
    targets[`${side}Weight`]=valid(wrist)&&torso>.08
      ? Math.max(0,Math.min(1,(landmarks[hip].y-landmarks[wrist].y)/(torso*.6))) : 1;
    targets[`${side}Arm`] = direction(shoulder, elbow);
    targets[`${side}ForeArm`] = direction(elbow, wrist);
    targets[`${side}Hand`] = direction(wrist, index) || direction(wrist, pinky);
  }
  return targets;
}
```

4.7 Sesión, cuenta atrás y pérdida del jugador

`session.js` coordina el inicio. Con la cámara activa, la cuenta atrás 3-2-1 no empieza hasta que la calibración sea válida (Figura 13). Si durante la partida se dejan de ver hombros o cadera por 10 resultados seguidos (unos 0,3 s), o si el detector no responde durante 1,2 s, el juego se pausa y vuelve a calibrar. Al recuperar al jugador se reinicia toda la cuenta. La Figura 14 muestra este flujo.

Figura 13

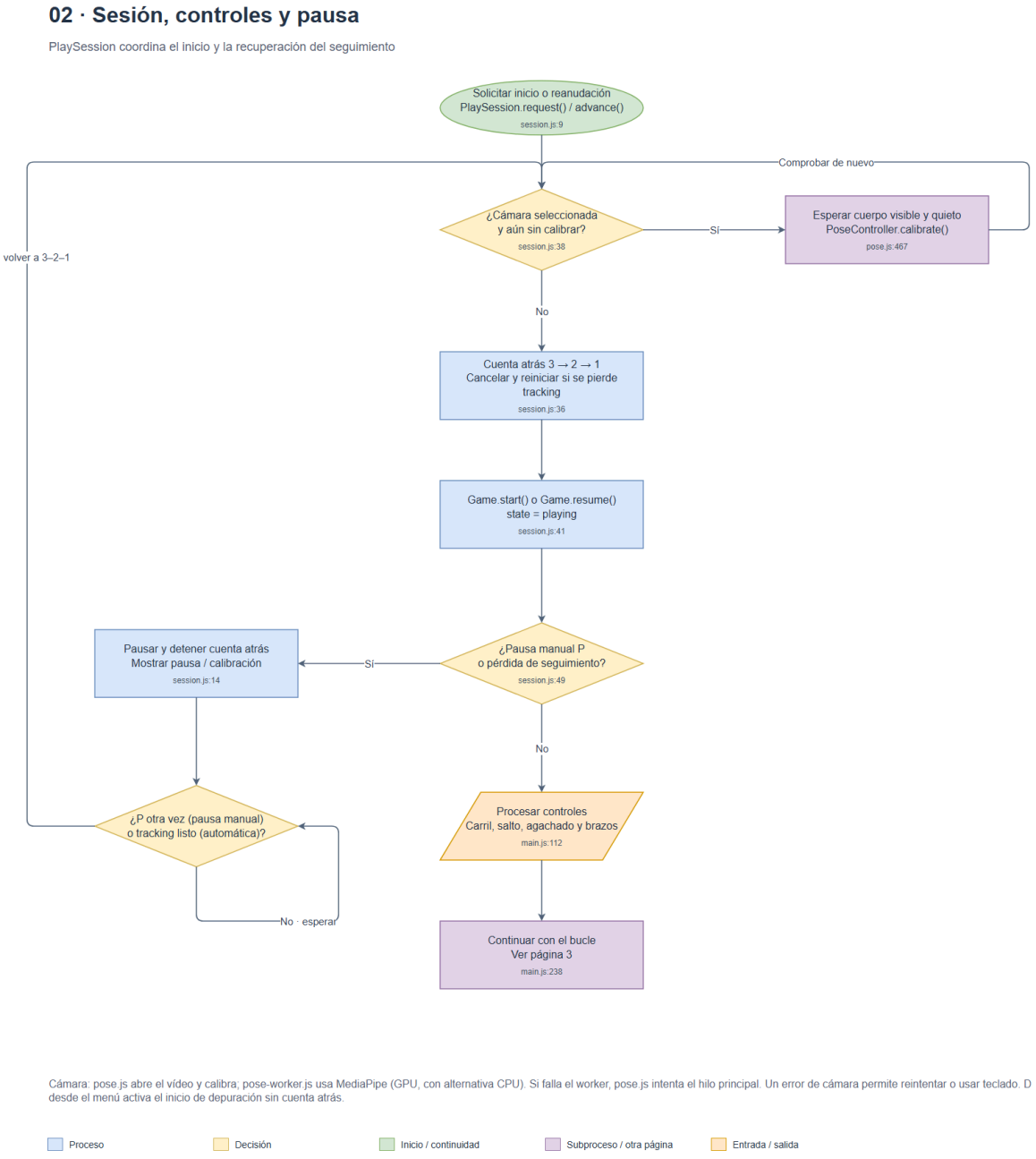
Cuenta atrás que espera la calibración (session.js).

session.js — líneas 36–51

```
  advance(n = 3) {
    this.cancel();
    if (!this.keyboard && this.camera && !this.ready) {
      this.show('▮▮ Calibrando', 'Necesito ver tus hombros y cadera con claridad'); return;
    }
    if (!n) {
      this.pending = false; this.hide();
      if (this.game.state === 'paused') this.game.resume(); else this.game.start();
      return;
    }
    this.show(String(n), '¡Prepárate!', true);
    this.tick = this.timer.set(() => this.advance(n - 1));
  }
  togglePause() {
    if (this.manual) { this.manual = false; this.pending = true; this.advance(); }
    else if (this.game.state === 'playing' || this.pending) {
```

Figura 14

Diagrama de flujo de sesión, controles y pausa (docs/diagrama-flujo/sesion.png).



4.8 Prueba práctica del seguimiento

Qué se quiso hacer. Comprobar que el seguimiento funciona dentro del juego completo, no solo en las pruebas unitarias, y obtener capturas de lo que ve el usuario.

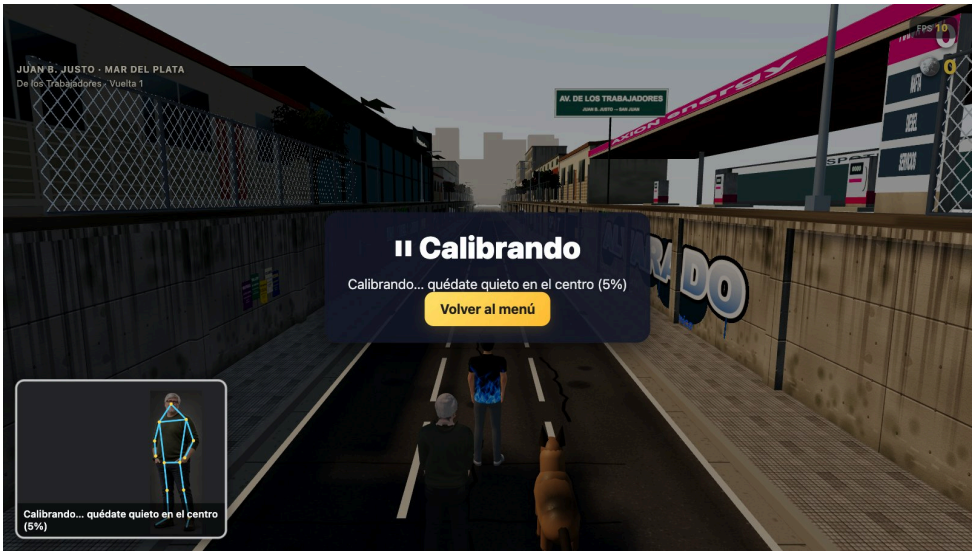
Qué se hizo. Como la prueba se automatizó sin una persona frente a la cámara, se usó una **cámara virtual**. Se armó un video de 22 segundos (640 × 480, 15 cuadros por segundo, formato Y4M) con la figura de cuerpo entero de la hoja de referencia de Estanga sobre fondo gris. La figura queda quieta 5 s, se desplaza a un costado y vuelve,

se desplaza al otro costado, sube y baja rápido (salto) y baja durante 3 s (agachado). Chrome lo usó como si fuera la cámara con las opciones `--use-fake-device-for-media-stream` y `--use-file-for-fake-video-capture`. Se abrió el juego, se activó la cámara desde Ajustes, se inició el modo infinito y se tomaron 40 capturas registrando el texto de estado del seguimiento.

Qué resultó. El detector arrancó en el worker con GPU (estado «worker GPU»). La Figura 15 muestra la pausa de calibración con el porcentaje de avance; la calibración se completó a los 23,4 s de haber activado la cámara, tiempo que incluye la carga del modelo en este equipo.

Figura 15

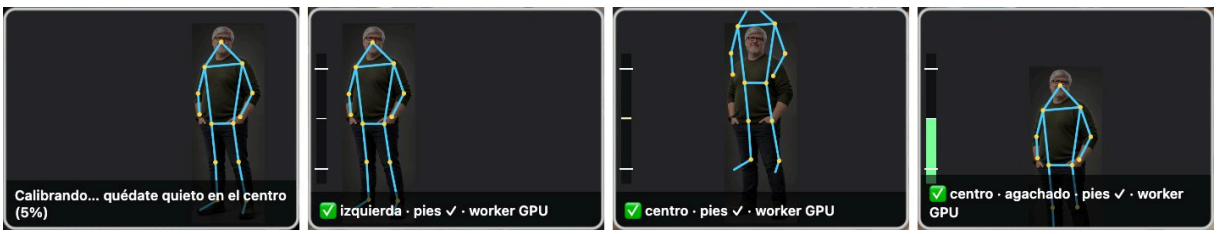
Juego en pausa mientras calibra: el esqueleto detectado aparece en el recuadro de la cámara.



En la Figura 16 se ven ampliados cuatro momentos del recuadro de cámara. El esqueleto celeste con puntos amarillos es el resultado de MediaPipe; la barra vertical de la izquierda es el medidor de salto (hacia arriba) y agachado (hacia abajo), y la raya blanca marca el umbral. El estado indica el carril detectado y si se ven los pies.

Figura 16

Recuadro de cámara durante la prueba: calibración, carril izquierdo, salto y agachado (la barra verde indica que se superó el umbral).



La Tabla 4 resume lo registrado. Los cambios de carril, el salto y el agachado del video fueron reconocidos y el juego reaccionó: se ve el aviso «SALTO» o «AGACHADO» y el personaje cambia de carril.

Tabla 4

Estados del seguimiento registrados durante la prueba con cámara virtual.

Tiempo (s)	Estado mostrado	Aviso del juego	Interpretación
23,4	✅ centro · pies ✓ · worker GPU	¡Listo! 🙌	Calibración completa
36,8	✅ derecha	➡	Desplazamiento lateral detectado
38,3 – 39,7	Sigo buscándote…	—	Pérdida momentánea del jugador
40,7	✅ centro	⬆	Vuelta al carril central
52,6	✅ izquierda	⬅	Desplazamiento al otro lado
57,1	✅ centro	SALTO ⤒	Subida rápida reconocida como salto
60,0 – 61,9	✅ centro · agachado	AGACHADO ⤓	Bajada sostenida reconocida
62,8	✅ centro	—	Salida del agachado

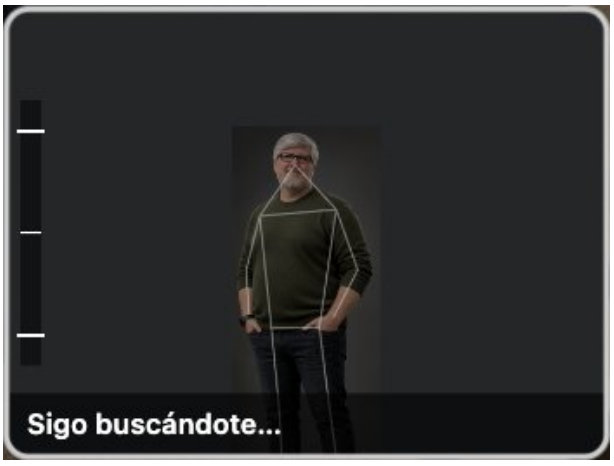
Nota. El tiempo se mide desde que se activó la cámara. Las capturas se tomaron cada 0,9–1,5 s porque cada captura de pantalla demoraba en este equipo; por eso la tabla muestra el primer registro de cada estado y no el instante exacto del gesto. El video se repite en bucle, así que los movimientos aparecen más de una vez.

Problemas observados. Después del primer desplazamiento lateral el estado pasó a «Sigo buscándote…» durante unos dos segundos (Figura 17). El código muestra ese mensaje cuando hay personas detectadas pero ninguna pasa los controles de `process`: tener hombros y caderas visibles dentro del cuadro y no haberse alejado demasiado de la posición anterior. Una posible causa es que en el video de prueba la figura se traslada de golpe (0,35 s) y se acerca al borde, cosa que una persona real no hace igual; no se comprobó cuál de los dos controles la descartó. El juego no se pausó porque el detector siguió entregando resultados y el jugador volvió antes de los 10 cuadros que activan la pausa.

También se observa en las capturas que el esqueleto a veces aparece corrido respecto del video (en el salto el esqueleto queda más arriba que la figura). El esqueleto se dibuja con el último resultado del detector, que llega algunos cuadros después de la imagen; cuando la figura se mueve rápido esa diferencia se hace visible.

Figura 17

Pérdida momentánea: hay una persona en la imagen pero el seguimiento la descarta.



Limitaciones. La prueba usa una imagen fija que se desplaza, no una persona que salta de verdad: la figura no dobla las rodillas ni cambia de forma. Sirve para comprobar que la cadena completa (cámara → worker → selección → calibración → gestos → juego) funciona, pero no reemplaza una prueba con una persona frente a una cámara física, que sigue pendiente de documentar con capturas propias. Las pruebas automáticas del repositorio también usan puntos simulados o el dispositivo de prueba de Chrome.

5. Jugabilidad, física y colisiones

El jugador corre siempre hacia adelante; lo que se mueve es el mundo. La velocidad empieza en 14 unidades por segundo y aumenta hasta 34. Hay tres carriles; el cambio de carril es una interpolación exponencial hacia la posición del carril elegido.

El salto usa integración balística: en cada cuadro la altura se actualiza con $y \mathrel{+}= vy \cdot dt - \frac{1}{2} \cdot g \cdot dt^2$, con gravedad 38 y velocidad inicial 13. Así la altura del salto es la misma aunque cambie la cantidad de cuadros por segundo. Si se pide un salto en el aire, se guarda 0,2 s y se ejecuta al aterrizar. El agachado dura 0,8 s y se prolonga mientras se mantenga la tecla o la postura (Figura 18).

Figura 18

Salto con memoria y agachado sostenido (game.js).

game.js — líneas 497–527

```
497  jump() {
498    if (this.state !== 'playing' || this.flyTimer > 0) return;
499    // Si aún estamos en el aire, el salto se guarda un momento y sale al aterrizar
500    if (!this.grounded) {
501      this.jumpBuffer = 0.2;
502      return;
503    }
504    this.jumpBuffer = 0;
505    this.vy = JUMP_SPEED;
506    this.grounded = false;
507    this.rollTimer = 0;
508  }
509  roll() {
510    if (this.state !== 'playing' || this.flyTimer > 0) return;
511    this.jumpBuffer = 0;
512    const entering = this.rollTimer <= 0;
513    this.rollTimer = Math.max(this.rollTimer, ROLL_TIME);
514    if (!this.grounded && entering) this.vy = Math.min(this.vy, -14); // en el aire: bajar de golpe
515  }
516  // Mientras sigas agachado delante de la cámara, el corredor sigue rodando
517  holdRoll() {
518    if (this.state !== 'playing' || this.flyTimer > 0) return;
519    this.rollTimer = Math.max(this.rollTimer, 0.25);
520  }
521
522  releaseRoll() {
523    this.rollTimer = Math.min(this.rollTimer, 0.12);
524  }
525
526  // --- Generación de obstáculos ---
527  spawnRow(z) {
```

Figura 19

Partida con teclado: salto (izquierda) y agachado frente a un banco (derecha).



Los obstáculos son tableros de dibujo (se saltan), bancos rotos (se pasa agachado) y colectivos de las líneas 542, 720 y 563, a los que se puede subir por una rampa y correr por el techo. Las colisiones se calculan en `collisions.js` con un barrido: en lugar de mirar solo la posición final del cuadro, se calcula en qué momento del cuadro el jugador entra y sale del volumen del obstáculo, tanto en el eje lateral como en el de avance (Figura 20). Esto evita que un obstáculo se «atraviese» cuando un cuadro tarda mucho.

Figura 20

Cálculo del intervalo de contacto con un obstáculo dentro del cuadro (collisions.js).

collisions.js — líneas 13–37

```
13 export function obstacleContact(obstacle, fromX, toX, targetX, dt, move) {
14   const { x, z } = obstacle.mesh.position;
15   const lateralTime = boundary => {
16     if (dt > 0 && fromX !== targetX) {
17       const ratio = (boundary - targetX) / (fromX - targetX);
18       if (ratio <= 0) return Infinity; // frontera más allá del destino
19       return -Math.log(ratio) / (LANE_RESPONSE * dt);
20     }
21     return (boundary - fromX) / (toX - fromX);
22   };
23   const lateral = interval(fromX, toX, x - CONTACT_HALF_WIDTH, x + CONTACT_HALF_WIDTH, lateralTime);
24   const halfDepth = obstacle.depth / 2 + PLAYER_HALF_DEPTH;
25   const forward = interval(z - move, z, -halfDepth, halfDepth,
26     boundary => (boundary - (z - move)) / move);
27   if (!lateral || !forward) return null;
28   const enter = Math.max(0, lateral[0], forward[0]);
29   const exit = Math.min(1, lateral[1], forward[1]);
30   if (enter >= exit) return null;
31   return { time: enter, exit, side: lateral[0] >= 0 && lateral[0] > forward[0] + 1e-7 };
32 }
33
34 export function overlapsObstacle(obstacle, x, zOffset = 0) {
35   return Math.abs(obstacle.mesh.position.x - x) < CONTACT_HALF_WIDTH &&
36     Math.abs(obstacle.mesh.position.z + zOffset) < obstacle.depth / 2 + PLAYER_HALF_DEPTH;
37 }
```

Las reglas de derrota son: chocar de frente un colectivo o un banco termina la partida; tropezar con un tablero o rozar un colectivo de costado acerca al profesor Estanga durante 5 segundos, y un segundo tropiezo en ese tiempo significa que te atrapa (Figura 21). Cada contacto se cuenta una sola vez. La Figura 22 resume estas decisiones.

Figura 21

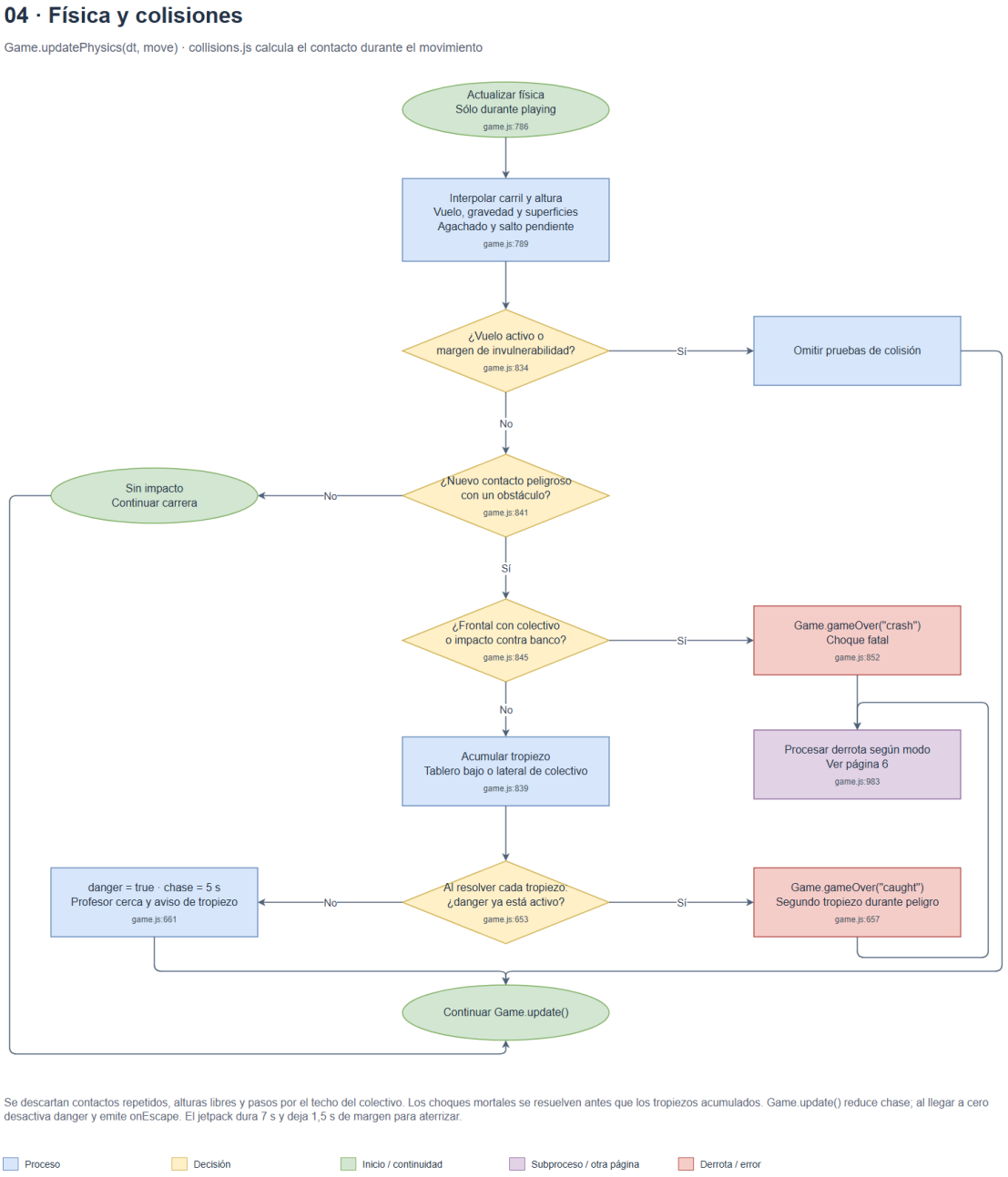
Tropiezo y captura por el profesor (game.js).

game.js — líneas 654–667

```
654   stumble(obstacle) {
655     if (this.state !== 'playing' || (obstacle && this.stumbleContacts.has(obstacle))) return;
656     if (obstacle) this.stumbleContacts.add(obstacle);
657     if (this.danger) {
658       this.gameOver('caught');
659       return;
660     }
661     this.danger = true;
662     this.chase = CHASE_TIME;
663     this.stumbleTimer = STUMBLE_TIME;
664     this.shake = 0.5;
665     this.cb.onStumble?.();
666     this.cb.onChase?.(this.chase / CHASE_TIME);
667   }
```

Figura 22

Diagrama de flujo de física y colisiones (docs/diagrama-flujo/colisiones.png).



La pausa (P o Esc) congela la física, las animaciones y la cámara, y ofrece Reanudar, Ajustes y Volver al menú (Figura 23). Al perder en el modo infinito se reproduce un video de derrota y después se ofrece guardar el puntaje con tres iniciales en un ranking local de diez posiciones (Figura 24). En la prueba se guardó la partida «AVE» con 436 puntos y 26 monedas, que quedó en el puesto 1 del ranking del navegador de prueba.

Figura 23

Menú de pausa con Esc.

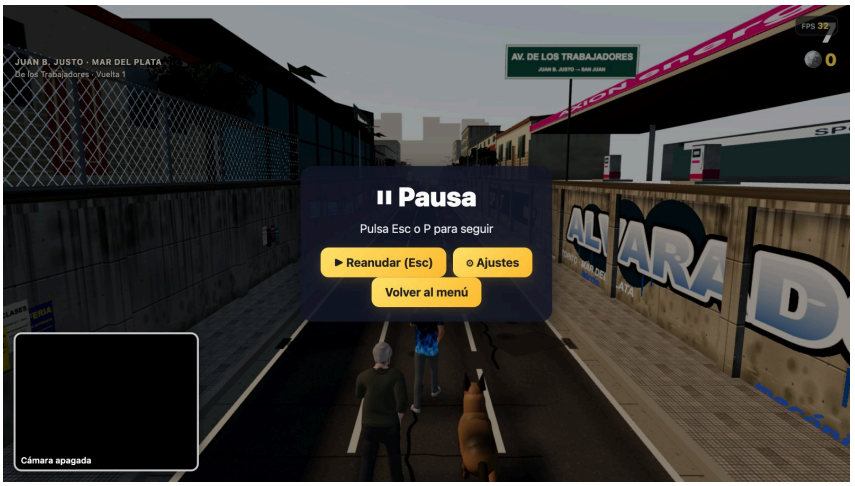
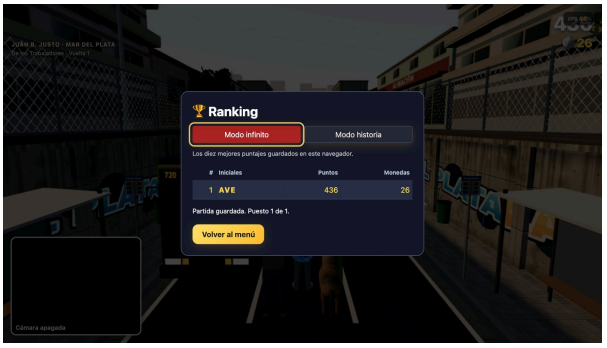


Figura 24

Final de la partida: video de derrota (izquierda) y ranking con el puntaje guardado (derecha).





El selector de calidad ajusta la resolución interna, el tamaño del mapa de sombras y la distancia de dibujado. En modo automático, cada 3 segundos se mide el tiempo medio por cuadro: si supera 22 ms (menos de unos 45 FPS) se baja la resolución un 12,5 %, y si es menor de 17 ms se sube.

6. Modo historia

El modo historia (`story.js`) encadena videos y tramos de carrera: video de introducción, ida desde la escuela hasta la casa recorriendo Juan B. Justo al revés, video en el que el alumno agarra el guardapolvo, vuelta hasta la E.E.T. N° 5 por el recorrido normal y video de victoria. Si choca, se reproduce el video de derrota y vuelve a empezar la ida con cero monedas. El puntaje es la cantidad de monedas juntadas. Las distancias y los videos están en una constante `STORY` (Figura 25); con los parámetros `?storyHome` y `?storySchool` en la URL se acortan los tramos para probar todo el flujo rápido, como se hizo para estas capturas.

Figura 25

Configuración del modo historia: tramos y videos (story.js).

story.js — líneas 12–28

```
12
13 export const STORY = Object.freeze({
14   // Ida: Juan B. Justo al revés, desde la escuela hasta la casa, junto a la estación Axion
15   // (el inicio del recorrido del modo infinito). Vuelta: el recorrido normal hasta la escuela.
16   homeAt: Number(query.get('storyHome')) || ROUTE_LENGTH - 2 * LOT_LENGTH,
17   schoolAt: Number(query.get('storySchool')) || 2004,    // frente a la E.E.T. Nº 5, último lote del recorrido
18   videos: Object.freeze({
19     intro: 'assets/videos/introduccion.mp4',
20     pickup: 'assets/videos/recoger-guardapolvo.mp4',
21     victory: 'assets/videos/final-victoria.mp4',
22     defeat: 'assets/videos/final-derrota.mp4',
23   }),
24   titles: Object.freeze({
25     intro: 'Intro', pickup: 'Agarrás el guardapolvo', victory: 'iLlegaste a tiempo!', defeat: 'No llegaste',
26   }),
27 });
28
```

Figura 26

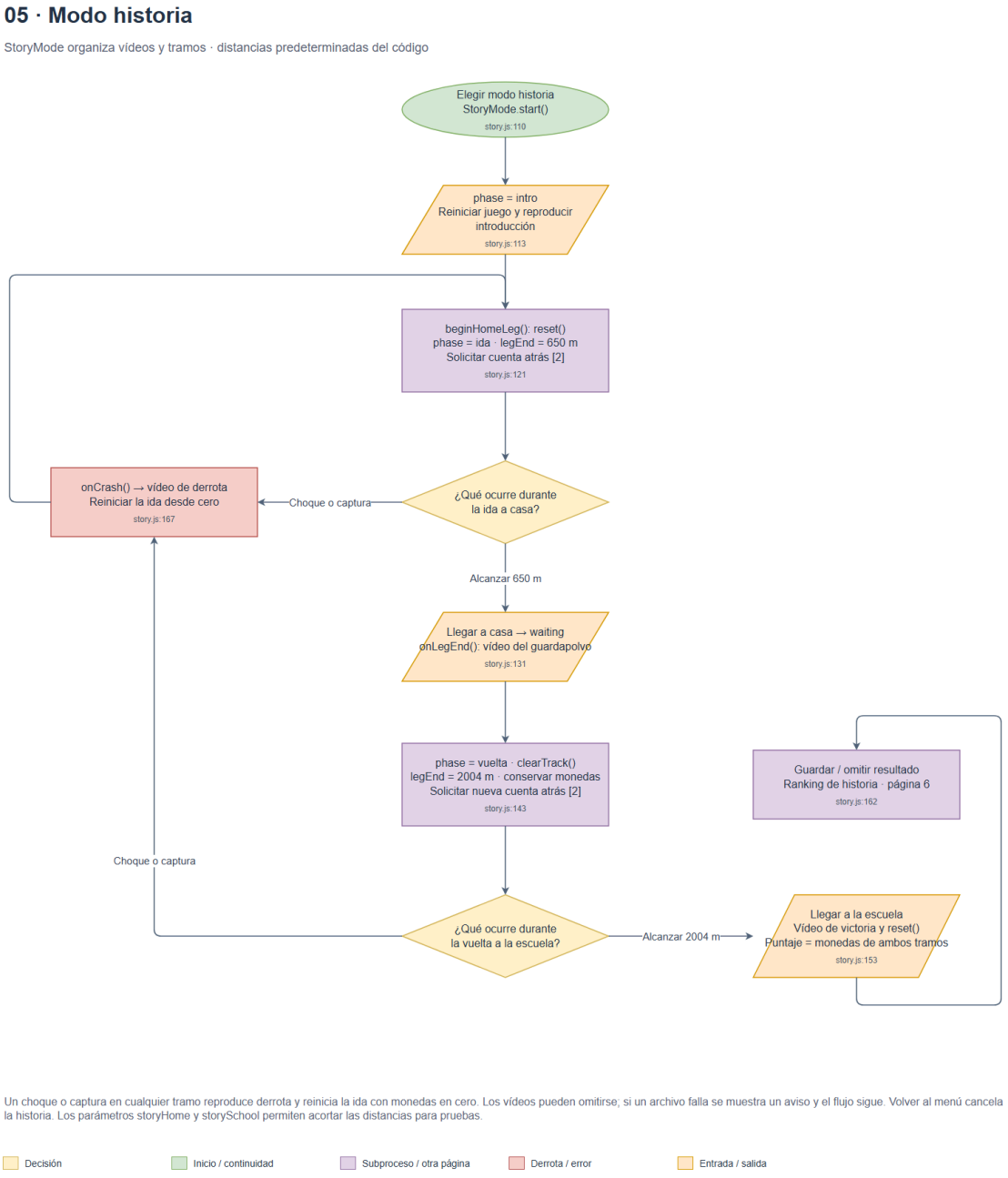
Modo historia: video de introducción, ida a casa con la barra de progreso y video de derrota con Estanga.



Cada video se puede saltar con Esc, Enter, Espacio o el botón SALTAR. Si falta un archivo de video, el juego muestra un cartel y sigue, para poder probar el recorrido aunque los videos no estén. La Figura 27 muestra el flujo completo.

Figura 27

Diagrama de flujo del modo historia (docs/diagrama-flujo/historia.png).



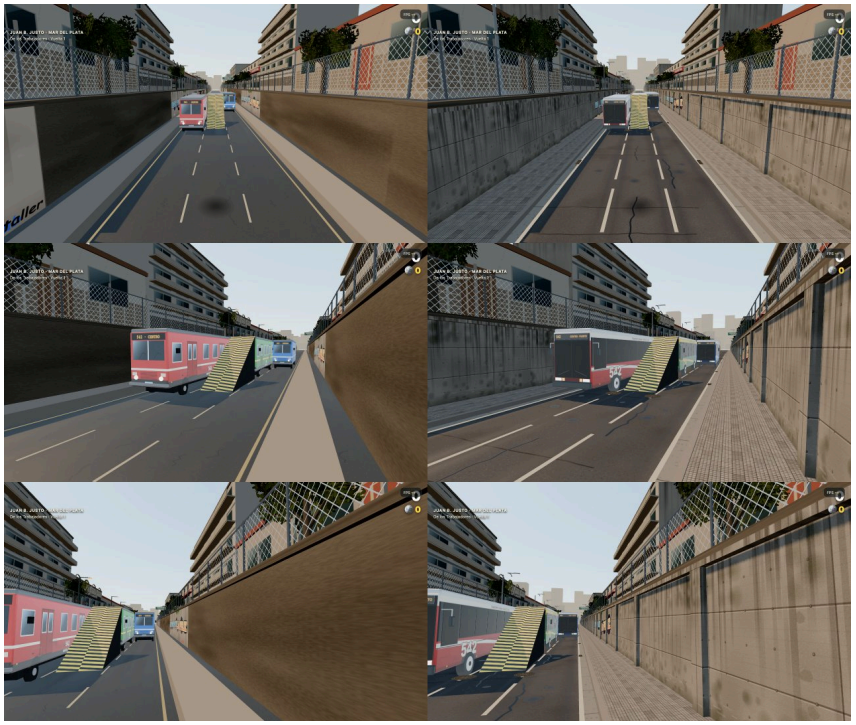
7. Escenario: Juan B. Justo

El recorrido va desde Av. de los Trabajadores hasta San Juan, en un bucle de 84 lotes por lado de 24 unidades (2016 unidades por vuelta). Los hitos (estaciones de servicio Axion, YPF y Shell, Bendu, la maderera, el Polideportivo Islas Malvinas y la Técnica 5) tienen geometría propia, no un mismo cubo con distinto cartel. Las texturas de asfalto, veredas, paredones de hormigón y murales (Aldosivi, Alvarado, SPOT, «La Feliz») se dibujan en el navegador con `canvas` y azar con semilla fija, así que el recorrido

se ve igual en cada partida y no se descargan fotos. Es una interpretación escenográfica: las distancias están comprimidas para el juego (Figura 28).

Figura 28

Comparación del escenario antes y después de la renovación de calle, veredas, paredones y colectivos (test-results/map-despues/antes-despues.png).



8. Modelado de los personajes

Los personajes son mallas 3D con esqueleto (*skinned meshes*) exportadas en formato glTF binario (GLB), que es el formato que carga Three.js. Cada personaje tiene dos niveles de detalle (LOD): uno alto para cerca y uno reducido. Todos comparten un esqueleto con nombres de huesos de Mixamo (`mixamorig*`); por eso la física, las animaciones procedurales y el seguimiento de brazos funcionan igual con cualquier personaje. La Tabla 5 resume el origen de cada uno.

Tabla 5

Origen y método de modelado de cada personaje.

Personaje	Origen de la malla	Esqueleto y pesos
El pibe	teen-boy de theskipper (Sketchfab, CC BY 4.0)	Rig Mixamo original reducido; pasa de pose T a pose A (`rig_detailed.py`)
El técnico	Anatomía MakeHuman (CC0) y guardapolvo modelado por código	Pesos anatómicos de MakeHuman (`build_classic.py`)
La del buzo / La conectada	Escaneos aportados por el grupo	Cuerpo MakeHuman ajustado al escaneo y transferencia de pesos (`fit_scan_rig.py`)
Estanga	Reconstruido desde una hoja de referencia	Pesos anatómicos de MakeHuman (`build_estanga.py`)

Nota. Fuente: docs/personajes-detallados.md y assets/characters/ATTRIBUTION.md. El esqueleto actual tiene 52 huesos (los 19 originales, clavículas, una vértebra intermedia y cinco dedos de tres falanges por mano).

8.1 Herramientas y flujo de trabajo

El modelado se hizo casi todo con scripts, para que se pueda repetir: Python con NumPy, OpenCV y SciPy para medir las referencias y generar la malla, y Blender 4.3 en modo sin interfaz (`blender -b --python …`) para el esqueleto, los modificadores y la exportación. La base anatómica es MakeHuman, que se publica con licencia CC0 y viene incluida comprimida en el repositorio. El flujo general fue:

1. Medir la referencia: recortar vistas, extraer siluetas y proporciones.
2. Deformar la base MakeHuman (edad, peso, proporciones) hasta que coincida con las siluetas.
3. Agregar ropa, pelo y accesorios inflando la superficie del cuerpo o modelándolos aparte.
4. Proyectar las fotos sobre las coordenadas UV para generar las texturas.
5. En Blender: crear el esqueleto, limitar los pesos, generar los dos niveles de detalle y exportar el GLB.
6. Comprobar en el juego con capturas automáticas y pruebas.

8.2 Estanga, el perseguidor

Estanga se reconstruyó a partir de una hoja de personaje (Figura 29) con vista de cuerpo entero, giro de frente, perfiles y espalda, primeros planos de la cabeza y detalles de anteojos, pelo, tejido, reloj, jean y botas.

Figura 29

Hoja de referencia usada para el modelado (assets/characters/source/estanga/reference-sheet.png).



Siluetas y proporciones. `estanga_reference.py` recorta las vistas y extrae máscaras. Con la altura total fijada en 1,80 m se obtienen la entrepierna (41 %), el dobladillo del suéter (51 %), el abdomen y la separación de piernas.

Rostro. `estanga_face_fit.py` usa MediaPipe Face Mesh, que detecta 478 puntos de la cara, tanto en el primer plano de la hoja como en un render de la cabeza 3D (Figura 30). Compara ambos conjuntos, escala la cabeza (× 1,14) y la deforma con *thin plate splines* en cuatro iteraciones hasta llegar a un error medio de 2,0 px (unos 2,4 mm). Luego corrige frente, nariz, labios y mentón con el perfil. El resultado se guarda en `face-fit.json`, así que reconstruir el modelo no necesita volver a correr MediaPipe (Figura 31).

Figura 30

Puntos faciales detectados sobre el primer plano frontal (test-results/estanga/ref/face_front_landmarks.png).



Figura 31

Ajuste iterativo del rostro: error en píxeles y escala de la cabeza (tools/estanga_face_fit.py).

tools/estanga_face_fit.py — líneas 104–129

```
104        l0, l1 = bb[px[:, 1], px[:, 0], 0], bb[px[:, 1], px[:, 0], 1]
105        bary = np.stack([l0, l1, 1 - l0 - l1], 1)
106        M3 = (P[allF[np.maximum(tri, 0)]] * bary[:, :, None]).sum(axis=1)
107        use = np.intersect1d(np.flatnonzero(ok), stable)
108        to_model = lambda pts: np.stack([-(pts[:, 0] * UP - view.u0) / S, (view.v0 - pts[:, 1] * UP) / S], 1)
109        Rm, Mm = to_model(R[use]), to_model(M[use])
110        err_px = np.linalg.norm(R[use, :2] - M[use, :2], axis=1)
111        fit['report'].append({'iteration': it, 'mean_error_px': float(err_px.mean()),
112                              'max_error_px': float(err_px.max())})
113        print(f'iteración {it}: error medio {err_px.mean():.2f} px, máx {err_px.max():.2f} px')
114        if it == 3:
115            # Cámara del primer plano: u = u0 - x·s, v = v0 - z·s (píxeles del recorte).
116            sel = use
117            u0 = float(np.mean(R[sel, 0] + M3[sel, 0] * em.S_FACE))
118            v0 = float(np.mean(R[sel, 1] + M3[sel, 2] * em.S_FACE))
119            fit['face_front_view'] = {'u0': u0, 'v0': v0, 's': em.S_FACE}
120            fit['reference_landmarks'] = R[:, :2].tolist()
121            break
122        if it == 0:
123            spread = lambda a: np.sqrt(((a - a.mean(axis=0)) ** 2).sum(axis=1).mean())
124            sigma = float(spread(Rm) / spread(Mm))
125            fit['scale'] = sigma
126            body.apply_face_fit({'pivot': pivot, 'neck_z': neck_z, 'scale': sigma, 'steps': []})
127            print('escala de cabeza', sigma)
128            continue
129        offset = (Rm - Mm).mean(axis=0)
```

Ropa, pelo y accesorios. El suéter y el jean se generan inflando la superficie del cuerpo hasta secciones medidas en las siluetas, con cuello acanalado, puños y pliegues. El pelo es una cáscara con dos capas de mechones con transparencia. Se

agregaron anteojos de acetato, barba, reloj inteligente en la muñeca izquierda y botas de trekking.

Texturas. `estanga_model.py` proyecta las vistas de la hoja sobre las UV, usando un z-buffer para saber qué parte de la malla ve cada foto, prioriza los primeros planos y borra los anteojos de las fotos de piel. Donde ninguna vista muestra nada (axilas, plantas) se usa piel de MakeHuman o el color medio de la prenda.

Esqueleto y niveles de detalle. `build_estanga.py` corre dentro de Blender. Crea el esqueleto, subdivide la ropa en el nivel alto, reduce la malla con el modificador *Decimate* en el bajo y limita cada vértice a 4 huesos, porque glTF admite como máximo 4 influencias por vértice (Figura 32).

Figura 32

Límite de 4 influencias por vértice y niveles de detalle en Blender (tools/build_estanga.py).

tools/build_estanga.py — líneas 112–147

```python
def limit_weights(obj, count=4):
    """glTF admite 4 influencias por vértice: conservar las mayores y normalizar."""
    vg = obj.vertex_groups
    for v in obj.data.vertices:
        gs = sorted([(g.weight, g.group) for g in v.groups], reverse=True)
        keep = gs[:count]
        total = sum(w for w, _ in keep) or 1
        for w, gi in gs[count:]:
            vg[gi].remove([v.index])
        for w, gi in keep:
            vg[gi].add([v.index], w / total, 'REPLACE')


def apply_lod(obj, setting):
    bpy.context.view_layer.objects.active = obj
    for o in bpy.context.selected_objects:
        o.select_set(False)
    obj.select_set(True)
    if setting == 'subdiv':
        bpy.ops.object.mode_set(mode='EDIT')
        bpy.ops.mesh.select_all(action='SELECT')
        bpy.ops.mesh.tris_convert_to_quads()
        bpy.ops.object.mode_set(mode='OBJECT')
        mod = obj.modifiers.new('Subdivision', 'SUBSURF')
        mod.levels = mod.render_levels = 1
        mod.uv_smooth = 'PRESERVE_BOUNDARIES'
        bpy.ops.object.modifier_move_to_index(modifier=mod.name, index=0)
        bpy.ops.object.modifier_apply(modifier=mod.name)
    elif setting < .999:
        mod = obj.modifiers.new('Decimate', 'DECIMATE')
        mod.ratio = setting
        mod.use_collapse_triangulate = True
        bpy.ops.object.modifier_move_to_index(modifier=mod.name, index=0)
        bpy.ops.object.modifier_apply(modifier=mod.name)
    limit_weights(obj)

```

El resultado dentro del juego se ve en la Figura 33. La validación medida se resume en la Tabla 6.

Figura 33

Estanga en el juego: reposo, primer plano, agarre y carrera (test-results/estanga/sheet_final.png).



Tabla 6

Validación del modelo de Estanga.

Medida	Resultado
Coincidencia de siluetas (IoU) frente / espalda	0,873 / 0,872
Coincidencia de siluetas (IoU) perfil izquierdo / derecho	0,852 / 0,905
Error medio del rostro (478 puntos)	2,0 px ≈ 2,4 mm
Triángulos nivel alto / bajo	80.060 / 28.753
Tamaño GLB alto / bajo	2,7 MB / 1,0 MB (más texturas compartidas)

Nota. Datos de docs/estanga.md. IoU (intersección sobre unión) vale 1 cuando la silueta del modelo y la de la referencia coinciden exactamente. Los valores de triángulos y del IoU corresponden al esqueleto de 19 huesos documentado en ese archivo.

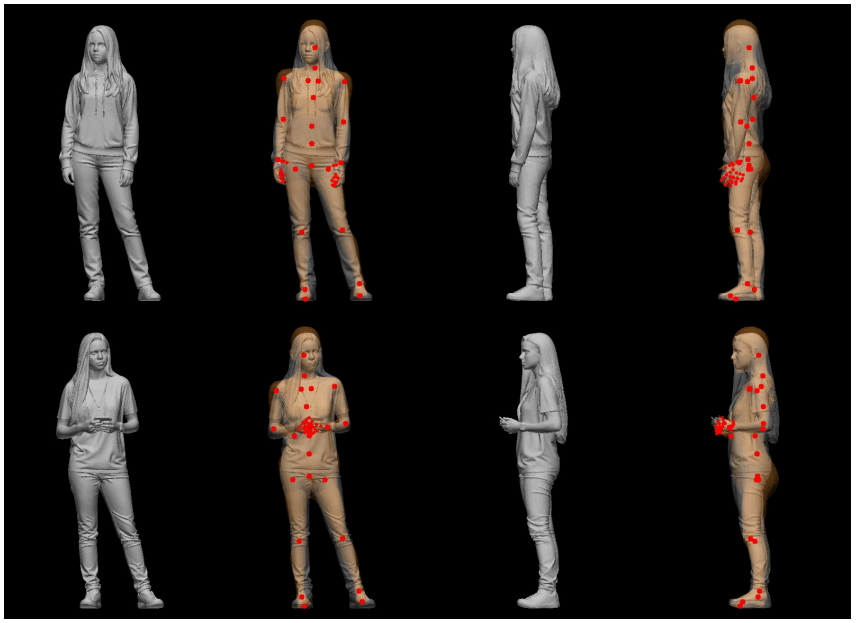
Límites. Es una reconstrucción desde una sola hoja de imagen, no un escaneo. Las zonas que ninguna vista muestra usan colores de relleno y parte de la luz de estudio quedó dentro de las texturas.

8.3 Personajes jugables

El pibe usa el modelo teen-boy con sus texturas originales; Blender normaliza escala y orientación, y adapta el esqueleto. Las dos chicas partían de escaneos sin esqueleto. Primero se les habían copiado las articulaciones de otro personaje y el codo quedaba fuera del brazo; la solución fue ajustar el cuerpo de MakeHuman a cada escaneo, moviendo cadera, columna, hombros, codos y rodillas hasta coincidir con la superficie, y recién después transferir los pesos (Figura 34). Las mallas se redujeron a unos 55 mil triángulos y un nivel bajo de unos 20 mil.

Figura 34

Ajuste del esqueleto a los escaneos: malla original, articulaciones ajustadas (rojo) y perfil (test-results/scans/fit_check.png).



Las animaciones son procedurales: se calculan por código en `character.js` y no vienen de archivos de animación. La carrera separa apoyo, despegue, fase aérea y recuperación, y la cadencia depende de la velocidad del juego. La Figura 35 muestra la hoja de control generada automáticamente para El pibe.

Figura 35

Ciclo de carrera, reposo, mano, salto, agachado y jetpack de El pibe (test-results/characters/technical-student/sheet.png).

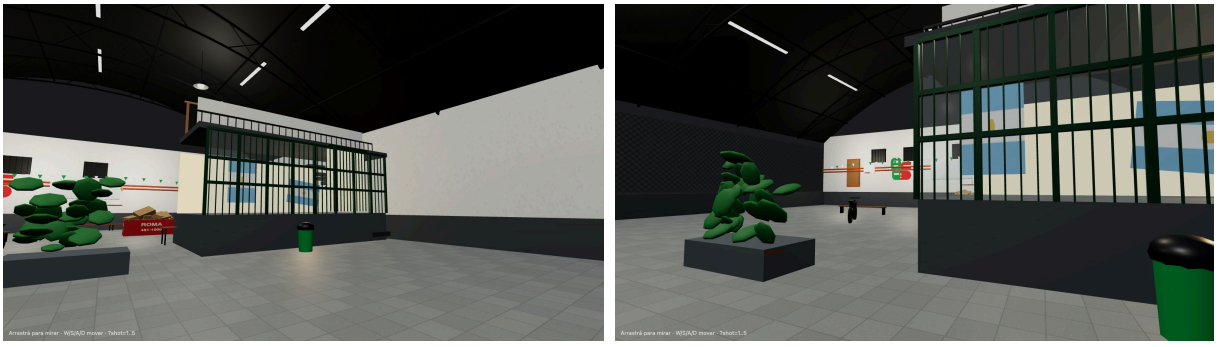


8.4 Interior de la escuela y casa

Para el modo historia se empezaron a modelar dos ambientes. `school-interior.js` arma el galpón de la Técnica 5 a partir de fotos de referencia: cabriadas en arco, tubos fluorescentes, piso de granito, cantina con rejas verdes y oficina en entrepiso (Figura 36). `tools/build_house.py` modela en Blender el living de la casa con el guardapolvo colgado. En la revisión documentada ninguno de los dos está integrado al juego: el interior se ve en `story-preview.html` y la casa genera renders aparte.

Figura 36

Vista previa del interior de la escuela (story-preview.html).



9. Sonido y música

Los efectos (botones, salto, moneda, tropiezo) no son archivos: se sintetizan con la Web Audio API usando osciladores de pocas notas. La música de fondo (`music.js`) lee la lista `assets/audio/playlist.json` y mezcla los temas con el algoritmo de Fisher-Yates: no repite un tema hasta haber pasado por todos y evita que el último suene dos veces seguidas (Figura 37). Un archivo dañado se saltea. Para no editar la lista a mano, un *hook* de Git (`.githooks/pre-commit`) ejecuta `tools/audio_playlist.py` en cada commit y agrega los MP3 nuevos.

Figura 37

Mezcla aleatoria de la lista de temas sin repeticiones (music.js).

music.js — líneas 76–95

```
76  pickNext() {
77    this.bag = this.bag.filter(track => !this.failed.has(track));
78    if (!this.bag.length) {
79      this.bag = this.tracks.filter(track => !this.failed.has(track));
80      for (let i = this.bag.length - 1; i > 0; i--) {
81        const j = Math.floor(this.random() * (i + 1));
82        [this.bag[i], this.bag[j]] = [this.bag[j], this.bag[i]];
83      }
84      const last = this.bag.length - 1;
85      if (last > 0 && this.bag[last] === this.current) [this.bag[0], this.bag[last]] = [this.bag[last], this.bag[0]];
86    }
87    this.audio.pause();
88    this.current = this.bag.pop() ?? null;
89    if (this.current) {
90      this.audio.src = this.current;
91      this.audio.currentTime = 0;
92      this.audio.loop = this.tracks.length - this.failed.size === 1;
93    }
94  }
95
```

10. Pruebas automáticas

El repositorio tiene pruebas con el ejecutor de pruebas de Node (`node --test`) y Playwright, que abre Chromium y juega de verdad: comprueba carga, puntuación, teclado, saltos, rampas, colectivos, jetpack, pausas, cámara con dispositivo simulado, ranking, música, modo historia y personajes. Por las limitaciones del equipo se ejecutaron en serie:

```
node --test --test-concurrency=1 tests/*.test.cjs
```

La ejecución completa duró 11 minutos y 23 segundos. De 68 pruebas, **61 pasaron y 7 fallaron**. Pasaron, entre otras, las de seguimiento de brazos, sesión y cuenta atrás, cámara con dispositivo simulado (con worker y con el respaldo sin worker), física de salto a 30, 60 y 120 Hz, techo de colectivos, agachado, colisiones, recorrido de Juan B. Justo y personajes. La Tabla 7 detalla las que fallaron.

Tabla 7

Pruebas que fallaron y causa observada.

Prueba	Error registrado	Causa
Lista MP3 (2 pruebas, audio-playlist.test.cjs)	`TypeError: write_text() got an unexpected keyword argument 'newline'`	Comprobada: `tools/audio_playlist.py` usa un parámetro que existe desde Python 3.10 y la Mac tiene Python 3.9.6.
Ranking desde el menú; indicador de persecución	`Cannot read properties of undefined (reading 'reversed')`	Comprobada: estas pruebas reemplazan `Game` por una versión simplificada sin `world`, y el último commit (b7ca369) agregó en main.js, línea 253, el acceso a `game.world.reversed`. En el juego real `world` existe y el error no apareció en las capturas.
Historia: intro, casa, regreso, victoria	Se esperaba 1 y se obtuvo 0	No determinada. Puede estar relacionada con el mismo cambio de recorrido invertido; no se investigó más.
Carga sin CDN; reproducción de MP3	Tiempo de espera agotado (30 s)	Posible causa: lentitud del equipo. Durante las capturas también hubo que esperar más de 30 s en el mismo clic. No se confirmó repitiendo la prueba aislada.

Nota. Resultados de `node --test --test-concurrency=1 tests/*.test.cjs` sobre la revisión b7ca369, con el Chromium de Playwright (renderizado por software SwiftShader).

11. Problemas encontrados y limitaciones

Durante la revisión del código para esta documentación se encontraron los siguientes puntos. Se informan sin corregir, porque el trabajo pedido era documentar.

- **Opciones de calidad sin efecto.** El menú ofrece «Ligera» y «Ultra», pero `Game.setQuality` solo acepta `auto`, `high` y `balanced` (game.js, línea 120). Al elegir esas dos opciones cambia el texto descriptivo pero el juego mantiene la configuración anterior.
- **Documentación desactualizada.** El README (línea 87) dice que MediaPipe y sus modelos se descargan de CDN/Google, pero desde el commit 759b169 se sirven desde `vendor/mediapipe`, como indica docs/menu-personajes.md.
- **Conflictos de merge que llegaron a main.** El commit 468a0cc tuvo que resolver marcadores de conflicto sin resolver en game.js y world.js, y el README conservó marcadores `<<<<<<< HEAD` hasta una revisión posterior. Conviene revisar `git diff --check` antes de subir.
- **Script de música incompatible con Python 3.9.** `tools/audio_playlist.py` usa `write_text(..., newline='\n')`, disponible desde Python 3.10. En equipos con Python 3.9, como el de prueba, el hook de commit falla. Además los scripts `music:index` y `setup:music` de package.json llaman a `python`, comando que no existe en macOS (solo `python3`).
- **Pruebas no actualizadas tras el último commit.** El acceso a `game.world.reversed` en main.js rompe dos pruebas que usan un juego simplificado (apartado 10).
- **Pérdida momentánea y retraso del esqueleto** en la prueba de seguimiento (apartado 4.8).
- **Ambientes del modo historia sin integrar.** El interior de la escuela y la casa existen como vista previa y script de Blender, pero el juego todavía reemplaza la casa con un video.
- **Parecido de los personajes.** Estanga se reconstruyó desde una sola hoja de imagen; no es un escaneo. Los dedos no se articulan en la animación aunque el esqueleto ya tenga huesos de dedos.
- **Rendimiento no garantizado.** En el equipo de prueba (GPU integrada Intel UHD 617) el contador mostró valores de entre 2 y 29 FPS durante las capturas, que además corrían junto al detector y a la herramienta de captura; no es una medición de rendimiento controlada.

Conclusión

Documentar el juego permitió ver que la parte más delicada no es dibujar la escena sino traducir un cuerpo en movimiento a tres acciones simples. El seguimiento funciona porque mide todo en proporciones del propio cuerpo, calibra antes de empezar, filtra el temblor sin agregar retraso y usa histéresis para no oscilar entre carriles. La prueba con cámara virtual confirmó que la cadena completa funciona dentro del juego, y también mostró sus puntos débiles: una pérdida momentánea del jugador y el retraso visible del esqueleto en movimientos rápidos.

En el modelado, el método de medir la referencia y ajustar por código hizo que los resultados se puedan repetir y comprobar con números (siluetas, error del rostro, cantidad de triángulos), aunque no reemplaza un escaneo real. Por último, la revisión encontró problemas concretos (opciones de calidad que no hacen nada, documentación desactualizada, un script incompatible con Python 3.9 y pruebas sin actualizar tras el último commit) y queda pendiente probar el seguimiento con una persona frente a una cámara física.

Referencias

Blender Foundation. (s. f.). *Blender 4.3 Manual*. https://docs.blender.org/manual/en/4.3/

Casiez, G., Roussel, N. y Vogel, D. (2012). 1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 2527–2530). ACM. https://doi.org/10.1145/2207676.2208639

Google AI for Developers. (s. f.). *Pose landmark detection guide*. MediaPipe Solutions. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker

Google AI for Developers. (s. f.). *Face landmark detection guide*. MediaPipe Solutions. https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker

Khronos Group. (2021). *glTF 2.0 Specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

MakeHuman Community. (s. f.). *MakeHuman* [Repositorio de código]. GitHub. https://github.com/makehumancommunity/makehuman

Mozilla. (s. f.). *MediaDevices: getUserMedia() method*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

Mozilla. (s. f.). *Using Web Workers*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers

Mozilla. (s. f.). *Web Audio API*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API

Playwright. (s. f.). *Getting started*. Microsoft. https://playwright.dev/docs/intro

theskipper. (s. f.). *teen-boy* [Modelo 3D, CC BY 4.0]. Sketchfab. https://sketchfab.com/3d-models/teen-boy-fff65cd5b0934eff8e3a281766a52bf5

three.js authors. (s. f.). *three.js documentation*. https://threejs.org/docs/

WebRTC Project. (s. f.). *Testing*. https://webrtc.github.io/webrtc-org/testing/